

LEADTAP AI – VECTOR BASED PROPERTY SEARCH SYSTEM

Developer – Gokulachalam.K

Tech Stack Used

1. Python + FastAPI

- Python
- FastAPI

2. Qdrant Vector Database (Docker) – Pulled qdrant image from docker

3. Embeddings (nomic-embed-text) – Pulled from Ollama

How It Understands User Queries

1. Semantic Understanding

- This system doesn't just look for keywords
- It understands the *meaning* behind what you type
- Example: "cheap" and "affordable" are treated similarly

2. Smart Query Parsing

- Extracts specific requirements from user text:
 - **Location:** "near Pine Street" → finds properties in that area
 - **Price:** "under 40k" → filters by budget
 - **Bedrooms:** "2 bedroom" → matches room count
 - **Amenities:** "parking" or "pet friendly" → checks features
 - **Preferences:** "nearby school" → prioritizes school proximity

3. Vector Search

1. **Converts your query** into a vector (mathematical representation)
2. **Finds similar property vectors** in the database
3. **Ranks results** by how closely they match your meaning

How Results Are Ranked

1. Base Similarity Score

- How well the property matches your query's meaning (0-1 scale)

2. Smart Boosting

- **Location Match:** +0.15 bonus if property is in your preferred area
- **Amenities:** +0.05 for each matching feature you want
- **School Proximity:** +0.1 if school is nearby (within 300m)

3. Final Ranking

- Combines similarity + boosts
- Shows you the most relevant properties first

Logic Explained

1. Natural Language Understanding

- Works with any query, not just specific formats
- "I need a quiet place near good schools" → understands the intent

2. Hybrid Approach

- Uses both semantic understanding AND structured filtering
- "2 bedroom under 40k" → understands both requirements

3. Explanations

- Tells you *why* each result was selected
- "Location match: Pine Street; nearby school 120m; base similarity: 0.82"

ARCHITECTURE DIAGRAM:

